

그린컴퓨터아카데미

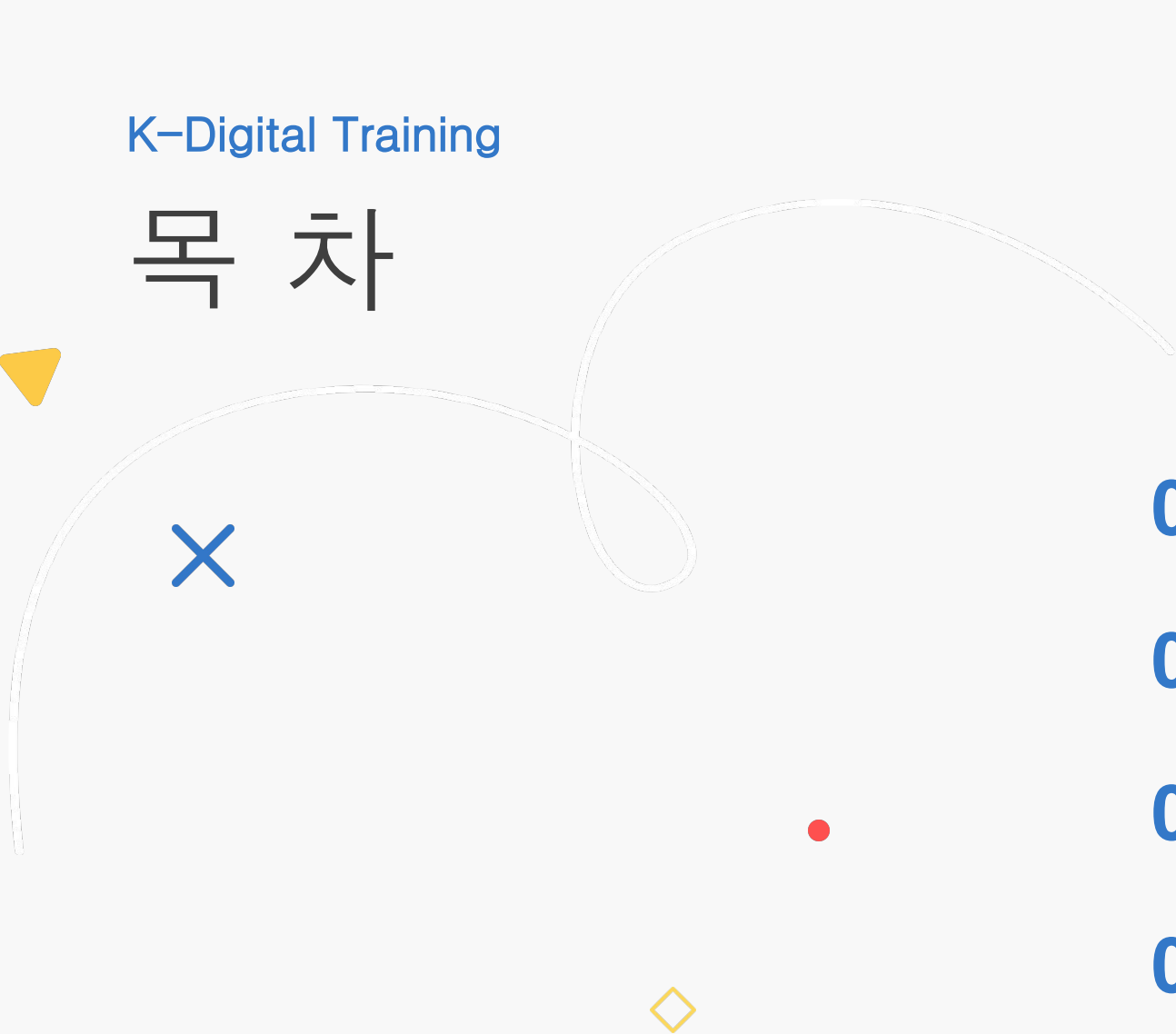
MarKit Place(AI 중고 거래 커뮤니티)

TEAM 2조 MarKit Place

손지윤, 양성빈, 조정우, 조총희,
유류진, 조현진, 황지백

[멘토] 신우철 대표님

목 차

- 
- 01 프로젝트 개요
 - 02 프로젝트 팀 구성 및 역할
 - 03 프로젝트 수행 절차 및 방법
 - 04 프로젝트 수행 경과
 - 05 자체 평가 의견

01

K-Digital Training

프로젝트 개요

▶ 프로젝트 주제 및 선정 배경, 기획의도

Markit Place

1. 프로젝트 주제 및 내용

Markit Place는 사용자들이 자신의 위치를 기반으로 중고 물품을 안전하고 편리하게 거래할 수 있도록 돕는 모바일 플랫폼입니다. 저희는 불필요한 물건에 새로운 가치를 부여하고 이웃 간의 소통을 활성화하여, 지속 가능한 소비 문화를 구축하는 것을 목표로 합니다.

프로젝트의 이름 '**Markitplace**'는 기술적 정체성과 사업적 확장성을 담고 있습니다. **Markit**은 GPS를 활용해 '위치를 표시하고 보상을 얻는' 핵심 기술을 직관적으로 표현하며, **Place**는 단순한 거래 장소를 넘어 사용자들이 모이는 커뮤니티 공간을 의미합니다. 이를 통해 향후 커머스를 넘어 다양한 소셜 서비스로 확장될 가능성을 열어두고 있습니다.

2. 선정 배경

지속 가능한 소비의 확산: 환경 문제에 대한 인식이 높아지면서 중고 거래는 불필요한 소비를 줄이고 자원을 재활용하는 **지속 가능한 소비 트렌드**로 자리 잡았습니다. 이 프로젝트는 사용자들이 경제적 이점과 환경 보호 가치를 동시에 얻도록 돕습니다.

지역 사회 연결성 강화: 비대면 시대에 '동네'의 중요성이 커지면서, 중고 거래는 이웃 간의 직접적인 교류를 촉진하는 매개체가 됩니다. **Markit Place**는 신뢰 기반의 거래를 통해 **지역 커뮤니티를 활성화**하는 데 기여합니다.

Flutter 기술 역량 강화: iOS와 Android를 하나의 코드로 개발할 수 있는 Flutter의 장점을 활용합니다. **효율적인 UI/UX 구축**과 **다양한 기능 구현**을 통해 Flutter 프레임워크에 대한 깊이 있는 이해와 실무 역량을 강화하고자 이 프로젝트를 선정했습니다.

01

K-Digital Training

프로젝트 개요

▶ 프로젝트 개발 환경

Markit Place

⚙️ 개발 환경

구분

Language

Framework

Build Tool

DBMS

ORM

Security

Back Git Repository

Front Git Repository

기술스택

Java 21

Spring Boot

Gradle

H2-console / sql

Spring Data JPA

Jwt 토큰 로그인 인증 처리

Market_place_server

Markit_place_front

🧩 사용 도구 및 개발 툴

항목

IDE

버전 관리

협업 도구

DB 테스트

라이브러리 관리

API 테스트 도구

VIEW

도구

IntelliJ IDEA / VsCode

Git / GitHub

Git Kanban / GitHub Issues / Notion

h2-Console

Gradle (groovy)

TalendAPI 및 Swagger

Flutter (SDK 3.27.4)

프로젝트 개요

주요 데이터 베이스 (Markit Place)

- CHAT_MESSAGE_TB
- COMMUNITY_POST_TB
- COMMUNITY_REPORT_TB
- ITEM_TB
- ITEM_REPORT_TB
- MEMBER_TB
- MEMBER_PROFILE_TB
- NOTICE
- PRAISE_TB
- TRADE_TB
- _TRADE_REVIEW_TB



02

K-Digital Training

프로젝트 팀 구성 및 역할

▶ 프로젝트 팀 구성 및 역할

훈련생	역할	담당 업무
조정우	팀장	 채팅,Flutter 레이아웃 작업 및 상태관리
손지윤	팀원	 커뮤니티 CRUD 및 좋아요, 정렬, 신고
조총희	팀원	 전역 예외처리 및 회원가입 로그인 인증, 레이아웃,소셜로그인
유류진	팀원	 판매상태,거래후기,레이아웃
양성빈	팀원	 AI 도입,SSE 서버 구축, 레이아웃, GPS 관련 기능
조현진	팀원	 공지사항,Q&A
황지백	팀원	 상품 CRUD 및 회원 신고 및 제재

03

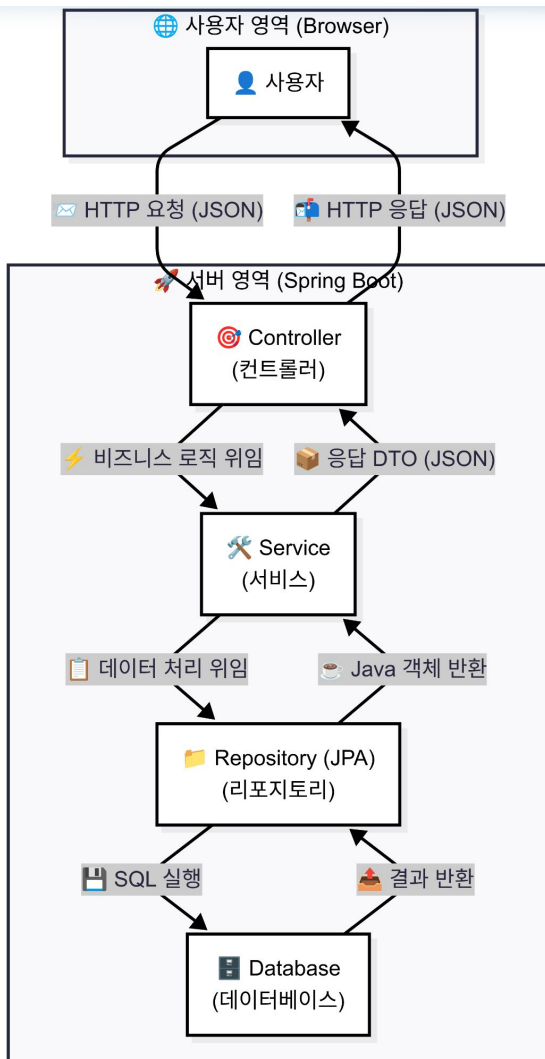
K-Digital Training

프로젝트 수행 절차 및 방법

▶ 프로젝트 일정 및 수행 절차 방법

구분	기간	활동		비고
사전 기획	8/25(월) ~ 8/26(화)	 프로젝트 기획 및 주제 선정	 기획안 작성	아이디어 선정
데이터 수집	8/27(수) ~ 8/29(금)	 필요 데이터 및 수집 절차 정의	 외부 데이터 수집	협약기업 데이터 협조
데이터 전처리	9/1(월) ~ 9/3(수)	 데이터 정제 및 정규화		프로그램 작업
모델링	9/4(목) ~ 9/17(수)	 전체적인 구현 및 정리		팀별 중간보고 실시
서비스 구축	9/17(수) ~ 9/24(수)	 마지막 점검 및 수정	 오류 수정	최적화, 오류 수정
총 개발기간	8/25(월) ~ 9/24(금)(총 4주)			

▶ 프로젝트 순서 및 요청 흐름

클라이언트 -서버 통신 및 시스템 구조

1. 클라이언트와 서버 간 통신

- **사용자**: Flutter로 만든 **모바일 앱**을 사용합니다.
- **요청 (HTTP Request)**: 사용자가 앱에서 버튼을 누르거나 데이터를 입력하면, Flutter가 HTTP 요청을 만들어 서버로 보냅니다. 이때, **RequestDTO** 형식으로 데이터를 **JSON** 바디에 담아 보냅니다.
- **응답 (HTTP Response)**: 서버는 처리 결과를 **ResponseDTO** 형식의 **JSON** 바디에 담아 Flutter 앱으로 반환합니다.

2. 서버 내부 동작 (Spring Boot)

- **Controller**: HTTP 요청을 받고, **RequestDTO**를 통해 클라이언트(Flutter 앱)가 보낸 **JSON** 데이터를 **Java** 객체로 변환합니다. 이후 비즈니스 로직 처리를 **Service** 계층에 위임합니다.
- **Service**: 비즈니스 로직을 수행합니다. 필요에 따라 **Repository**를 통해 데이터베이스에 접근합니다.
- **Repository**: JPA를 사용하여 데이터베이스에 **SQL**을 실행합니다. **save()**, **findById()** 등의 메서드를 통해 엔티티를 영속성 컨텍스트에 관리하거나 데이터베이스에 직접 반영합니다.
- **데이터베이스 (Database)**: 실제 데이터가 저장되는 곳입니다.

REST API 구조에서는 Controller가 Model에 데이터를 담아 View로 전달하는 과정이 생략됩니다.

대신 Controller가 Service로부터 받은 결과를 **ResponseDTO**로 변환하여 바로 클라이언트(Flutter 앱)에게 **JSON** 형태로 응답합니다. Flutter 앱은 이 **JSON** 응답을 받아 앱 화면에 필요한 정보를

프로젝트 수행 절차 및 방법

▶ Markit Place: 기대 효과 및 가치

🎯 지속 가능한 소비의 실현:

- 사용자들이 불필요한 물건을 판매하거나 필요한 물건을 합리적인 가격에 구매함으로써 자원 낭비를 줄이고 환경 보호에 기여하는 가치를 제공합니다.

🎯 지역 사회 연결성 강화:

- 근거리를 기반으로 한 직접 거래를 통해 이웃 간의 소통을 활성화하고, 신뢰를 바탕으로 하는 건강한 지역 커뮤니티를 형성합니다.

🎯 사용자 신뢰도 향상 및 재방문 유도:

- 매너 평가, 거래 후기 등 신뢰성 있는 데이터를 통해 사용자들의 활발한 참여를 유도합니다. 이는 플랫폼에 대한 만족도와 충성도를 높여 지속적인 재방문을 이끌어냅니다.

▶ 주요 업무 및 성과

🎯 핵심 목표

1:1 실시간 채팅 기능 구현: 실시간으로 메시지를 주고받는 채팅 시스템을 구축하여 사용자들이 즉각적으로 소통할 수 있도록 합니다.

🎯 핵심 목표

거래 약속 및 가격 협상 지원: 채팅 기능을 통해 사용자들이 거래 장소, 시간, 가격 등 구체적인 거래 조건을 편리하게 협의할 수 있게 합니다.

➡ 웹소켓(WebSocket) 기반 통신 시스템 구축:

WebSocket 기술 적용: 기존 HTTP와 달리 한 번 연결되면 지속적인 양방향 통신이 가능한 WebSocket을 사용해 실시간 채팅 환경을 마련했습니다.

➡ STOMP(Simple Text Oriented Messaging Protocol)

활용: 복잡한 웹소켓 메시지를 효율적으로 관리하기 위해 STOMP 프로토콜을 도입하여 메시지 라우팅을 체계화했습니다.

STOMP 엔드포인트 및 메시지 브로커 설정:

/ws-stomp 엔드포인트 생성: 클라이언트가 서버에 연결할 수 있는 통로인 **/ws-stomp** 엔드포인트를 만들었습니다.

메시지 브로커(MessageBrokerRegistry) 설정:

/topic과 **/queue**를 구독하는 메시지 브로커를 통해 메시지를 효율적으로 배포하고, **/app**으로 시작하는 메시지만 처리하도록 설정하여 서버 부담을 줄였습니다.

인증 및 보안 강화:

JwtHandshakeInterceptor 도입: 채팅 연결 시 JWT(JSON Web Token)를 인터셉터로 활용하여 사용자의 신원을 검증함으로써, 보안성이 높은 채팅 환경을 구축했습니다.

setAllowedOriginPatterns("*") 설정: CORS 문제를 해결하여 다양한 도메인에서 접속이 가능하도록 유연성을 확보했습니다.

브라우저 호환성 확보:

SockJS 라이브러리 사용: 웹소켓을 지원하지 않는 구형 브라우저에서도 채팅 기능이 작동할 수 있도록 SockJS를 추가하여 서비스의 접근성을 높였습니다.

▶ WebSocket 동작원리

→ 1. WebSocket 연결

- 채팅 메시지를 보낼 경우
- 클라이언트가 api/ws-stomp 엔드포인트를 연결 시도 할 시에
JwtHandshakeInterceptor 가 요청에 포함된 JWT 검증은 한 후에 userId를 세션에 저장

→ 2. 클라이언트가 메시지 전송

- STOMP를 통해 메시지를 보낼 때 /app/chat/sendMessage 를 타고 들어가서 Service
메서드를 실행 시킴

→ 3. 서버에서 메시지 처리

- 세션에서 사용자 ID 가져오기 -> 인증된 유저가 보낸 메시지만 처리
- 채팅방이 존재하면 -> 기존 방에서 메시지 저장
- 채팅방이 없으면 -> 새 방 생성 후 메시지 저장

→ 4. 메시지 전송 (브로드캐스트)

- 메시지가 저장되면 해당 브로커를 구독하고 있는 사람들에게 모든 유저들에게 메시지를
보여줍니다

```
@Override no usages 👤 whwwwjddn123 *  
public void registerStompEndpoints(StompEndpointRegistry registry) {  
    registry.addEndpoint( ...paths: "api/ws-stomp").setAllowedOriginPatterns("*")  
        .addInterceptors(jwtHandshakeInterceptor)  
        .withSockJS();  
}
```

```
@PostMapping("/chat/sendMessage") no usages 👤 whwwwjddn123  
public void sendMessage(@Payload ChatMessageRequestDTO.Message msgDTO,  
    SimpMessageHeaderAccessor headerAccessor) {
```

```
Long senderId = (Long) headerAccessor.getSessionAttributes().get("userId");  
  
ChatMessageResponseDTO.MessageDTO responseDTO =  
    chatMessageService.saveAndProcessMessage(senderId, msgDTO);
```

```
messagingTemplate.convertAndSend( destination: "/topic/chat/room/" +  
    responseDTO.getRoomId(), responseDTO);
```

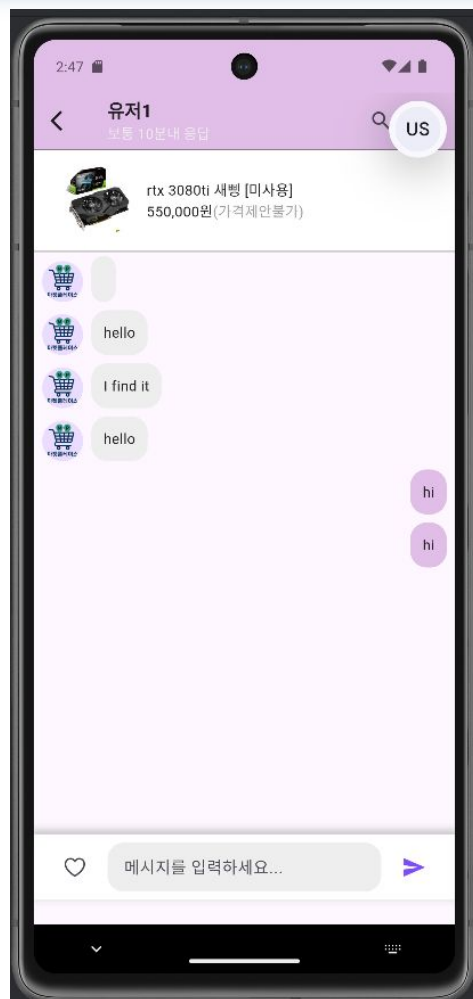
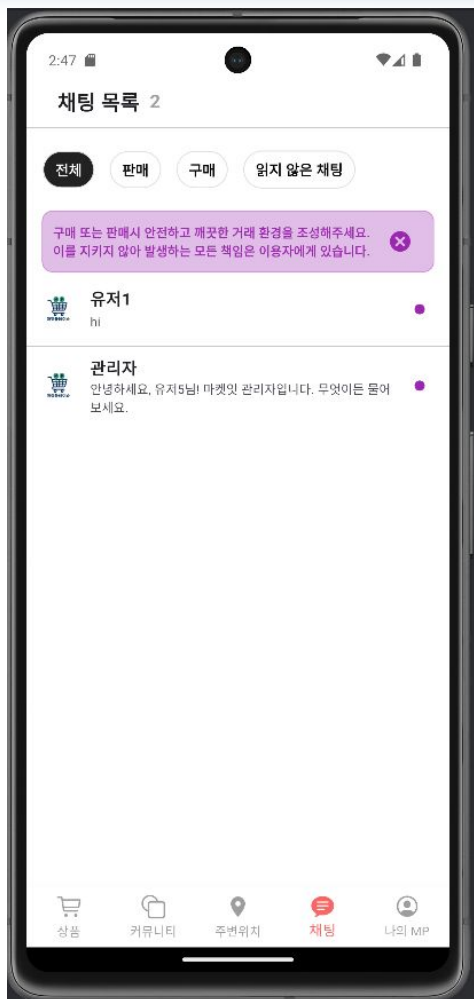
04

K-Digital Training

WebSocket 1:1 실시간 메시지 구현 영상

▶ 주요 업무 및 성과

- 채팅방 리스트 화면
- 채팅방 화면
- 전체적인 구현 영상



04

K-Digital Training

프로젝트 수행 경과

▶ 주요 업무 및 성과

거래 상태 관리: 판매자가 상품 상태를 명확하게 표시하고, 구매자도 이를 쉽게 확인할 수 있도록 합니다.

신뢰 기반 평점 시스템: 거래 후기를 통해 판매자와 구매자 간의 신뢰도를 높입니다.

고도화된 검색 및 필터링: 사용자가 원하는 상품을 빠르고 정확하게 찾을 수 있도록 검색 기능을 개선합니다.

판매 상태 관리 기능: 상품의 판매 상태를 '판매중', '예약중', '거래완료'의 세 가지로 나누어 관리합니다. 이를 통해 판매자는 상품의 현재 상태를 효율적으로 업데이트하고, 구매자는 불필요한 문의를 줄일 수 있게 됩니다.

거래 후기 시스템 구현

평점 부여: 거래가 완료되면 상대방에게 '평점'을 남길 수 있는 기능을 추가했습니다.

점수 산정: 평점은 초기 점수 0점에서 긍정적인 평가를 받을 때마다 점수가 0.5점씩 올라가는 **+0.5점 구조**로 설계하여 사용자의 긍정적 거래 경험을 장려합니다.

다양한 필터 기능: 사용자가 원하는 상품을 쉽게 찾을 수 있도록 **가격 범위, 카테고리별** 필터링 기능을 추가했습니다.

키워드 검색 기능: 상품명이나 설명에 포함된 특정 **키워드**를 검색하여 관련 상품을 찾아볼 수 있도록 기능을 구현했습니다.

정렬 기능: 검색 결과에 대해 **인기순**과 **최신순**으로 **정렬**하는 기능을 제공하여 사용자가 선호하는 순서대로 상품을 탐색할 수 있게 했습니다.

▶ 주요 업무 및 성과

List<String> 형태의 칭찬 주제 목록을

입력 받아서 텍스트를 생성하는 메서드

1. 토픽이 없을 때 : 만약 입력된 주제 목록이 비어있거나 null 이라면 “좋은 거래 감사합니다” 라는 기본 메시지를 반환
2. 토픽이 하나일 때 : 목록에 주제가 하나 있다면, “다음과 같은 이유로 칭찬 드려요 첫번째 주제와 같은 형식으로 텍스트를 만들어줌
3. 토픽이 여러 개일 때 : 목록에 주제가 두 개 이상 있다면, 마지막 주제 앞에 "그리고"를 붙여 자연스러운 문장을 만듭니다.
 - 예시: ["친절해요", "시간을 잘 지켜요"]는 "다음과 같은 이유로 칭찬 드려요: 친절해요 그리고 시간을 잘 지켜요."로 변환됩니다.
 - 예시: ["친절해요", "시간을 잘 지켜요", "답변이 빨라요"]는 "다음과 같은 이유로 칭찬 드려요: 친절해요, 시간을 잘 지켜요 그리고 답변이 빨라요."로 변환

```
public class PraiseContentGenerator { 1 usage  👤 yooryujin *  
  
    private static final ResourceBundle BUNDLE = ResourceBundle.getBundle( baseName: "messages", Locale.KOREA); 3 usages  
  
    public static String generateContentFromTopic(List<String> topicNames) { 1 usage  👤 yooryujin *  
  
        String defaultPraise = BUNDLE.getString(PraiseMessage.DEFAULT.getKey());  
        String prefix = BUNDLE.getString(PraiseMessage.PREFIX.getKey());  
        String andSeparator = BUNDLE.getString(PraiseMessage.SEPARATOR.getKey());  
  
        if (topicNames == null || topicNames.isEmpty()) {  
            return defaultPraise;  
        }  
  
        if (topicNames.size() == 1) {  
            return prefix + topicNames.get(0) + ". ";  
        }  
  
        String result = String.join( delimiter: ", ", topicNames.subList(0, topicNames.size() - 1)  
            + andSeparator + topicNames.get(topicNames.size() - 1);  
        return prefix + result + ". ";  
    }  
}
```


04

K-Digital Training

프로젝트 수행 경과

▶ 주요 업무 및 성과

1. 후기 생성 : **tradeId**와 **reviewerId**를 받아 데이터베이스에서 해당 거래와 사용자를 찾습니다. 만약 존재하지 않으면 예외를 발생
2. 후기 작성 권한 검증 : **validateReviewer** 메서드를 호출하여 후기를 작성하는 사람이 **구매자 인지 확인**하고, 이미 후기를 작성했는지 중복 여부를 체크
3. 후기 저장 : 유효성 검사를 통과하면, **TradeReview** 객체를 생성하여 데이터베이스에 저장
4. 후기를 작성하려는 **reviewer**가 해당 거래의 **buyer** 인지 확인
5. 중복 후기나 권한이 없는 경우, 각각 **Exception404** 예외를 발생시켜 작업을 중단

// 후기 작성 권한 검증 및 중복 체크

```
private void validateReviewer(Trade trade, Member reviewer) { 1 usage 2 yoorujin *  
    if (reviewer.getId().equals(trade.getBuyer().getId())) {  
        if (trade.isBuyerReviewed()) {  
            throw new Exception400("구매자 후기는 이미 작성되었습니다.");  
        }  
        trade.setBuyerReviewed(true);  
    } else {  
        throw new Exception403("후기를 작성할 권한이 없습니다.");  
    }  
}
```

04

K-Digital Training

프로젝트 수행 경과

▶ 주요 업무 및 성과

🎯 핵심 목표

커뮤니티 활성화

중고거래 플랫폼 내에서 사용자들이 자유롭게 소통할 수 있는 공간 제공합니다.

주제별 카테고리(대분류/세부 분류)를 통한 체계적인 게시글 관리합니다.

참여도 향상

게시글 및 댓글 작성 기능을 통해 사용자 간 상호작용 촉진합니다.

좋아요 기능으로 콘텐츠 선호도 반영 및 커뮤니티 활성화 강화합니다.

신뢰성 확보

게시글에 대한 신고 기능을 제공하여 부적절한 콘텐츠 제재 가능하도록 합니다.

🔧 주요 기술 설명

게시글/카테고리 관리: @ManyToMany, @OneToMany 매핑으로 게시글, 카테고리, 구조 설계 및 **CRUD** 구현

좋아요: @OneToMany 매핑, 게시글 좋아요·댓글 좋아요 관리

정렬 지원: 댓글 최신순·등록순 정렬 제공

신고 프로세스: 신고 등록 시 중복 방지, 사유 유효성 검증, 상태관리

소유권 및 권한 검증: @Auth 어노테이션으로 접근 제어

영속성 & 최적화: CascadeType.ALL 로 연관 데이터 자동 삭제

04

K-Digital Training

프로젝트 수행 경과

▶ 주요 업무 및 성과

🔧 처리 흐름

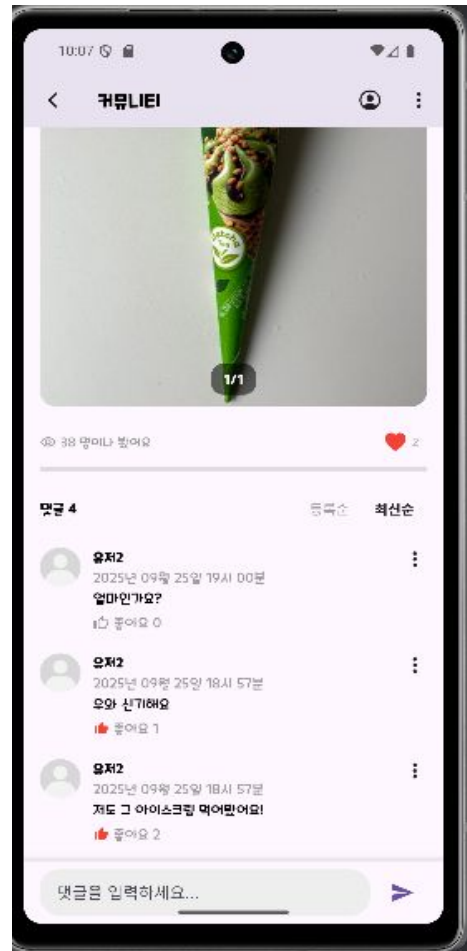
게시글과 댓글 좋아요

클라이언트 요청: 사용자가 게시글 또는 댓글의 좋아요 버튼 클릭 상태 토글 방식 (좋아요 추가 / 취소)

Controller 처리: 로그인된 사용자 검증, 어떤 게시글/댓글에 대한 좋아요 요청인지 확인

Service 처리: 이미 좋아요 한 상태인지 체크
좋아요 안한 상태 → 좋아요 추가 / 좋아요 한 상태 → 좋아요 취소

응답 반환: 좋아요 상태(눌렀는지 여부)와 현재 좋아요 개수 반환
클라이언트는 이를 반영하여 UI 업데이트



04

K-Digital Training

프로젝트 수행 경과

▶ 주요 업무 및 성과

🔧 처리 흐름

게시글 댓글 정렬

클라이언트 요청: 사용자가 게시글 상세 화면 진입 → 댓글 정렬 옵션 클릭

Controller 처리: 게시글 ID와 정렬 옵션을 파라미터로 Service 계층에 전달

Service 처리: 게시글에 속한 댓글 조회 → 정렬 기준에 따라 데이터 가공 (등록순, 최신순)

응답 반환: 정렬된 댓글 리스트를 반환, 클라이언트는 이를 기반으로 UI에 댓글 목록을 렌더링



04

K-Digital Training

프로젝트 수행 경과

▶ 주요 업무 및 성과

🔧 처리 흐름

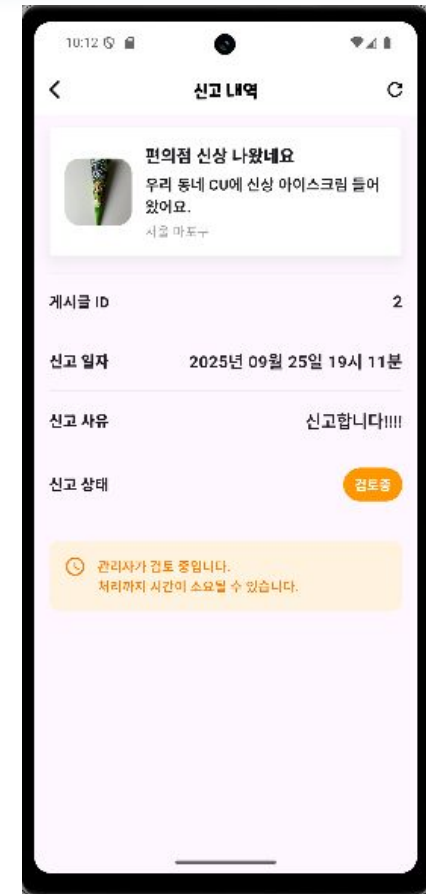
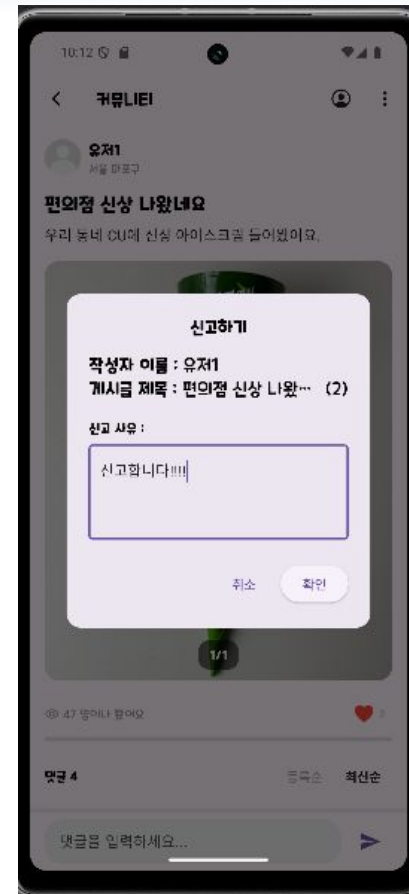
게시글 신고하기

클라이언트 요청: 사용자가 게시글에서 “신고하기” 버튼 클릭 → 신고 사유(reason) 입력 후 요청

Controller 처리: 로그인된 사용자 검증 후 요청을 Service로 전달

Service 처리: 작성자와 신고 대상 게시글 조회
→ 동일 사용자가 이미 신고한 게시글인지 체크
→ 기본값 PENDING 상태로 적용

응답 반환: 신고내역 UI에서 접수된 신고 목록 렌더링



04 공용모듈, 회원(Member) 관련 기능 구현(백엔드)

핵심 목표

공용 모듈 개발

팀원들이 원활하게 **CRUD** 기능을 구현하고
테스트해볼 수 있는 공용 인프라 구축

멤버 가입, 인증을 위한 **CRUD**

회원가입 (약관동의 » 중복검사 » 이메일인증 » 회원가입)

로그인 (소셜계정//이메일//아이디 선택)

계정 (아이디찾기, 비밀번호재설정)

프로필관리 (프로필수정, 탈퇴)

상태관리 (조회, 승급, 정지, 삭제)

빠른 개발속도 필요

04 공용모듈, 회원(Member) 관련 기능 구현(백엔드)

핵심 목표

공용 모듈 개발

CRUD 기능구현을 위한 공용 인프라

가입, 인증을 위한 CRUD

회원가입

로그인

프로필관리

상태관리

기술적 특징

커스텀 인증/인가

Interceptor를 활용한 직관적인 보안 모델 구현 (AuthInterceptor, @Auth)

JWT 기반 토큰 시스템

Stateless 서버를 위한 토큰 라이프사이클 관리 (JwtUtil, HttpOnly Cookie)

전역 예외 처리 및 표준 응답

일관된 API로 클라이언트 개발 편의성 증대 (@RestControllerAdvice, ApiUtil)

순서 제어 기반 데이터 초기화

@Order를 통한 안정적인 테스트 데이터 구축 (CommandLineRunner, @Order)

비동기 이메일 발송

@Async를 통한 API 응답 속도 최적화 (@Async, JavaMailSender)

04 계정 회원가입

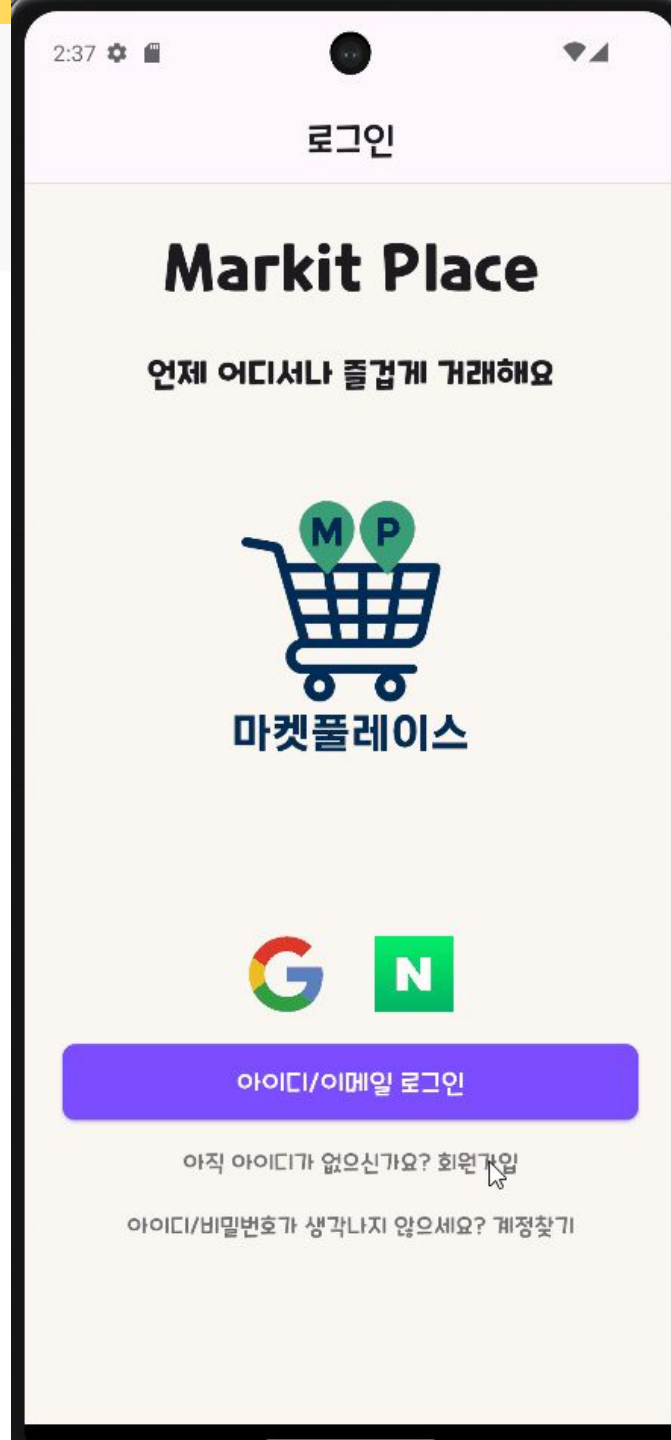
약관동의

아이디 중복확인

비밀번호 유효성검사 / 확인

이메일 인증 (중복검사)

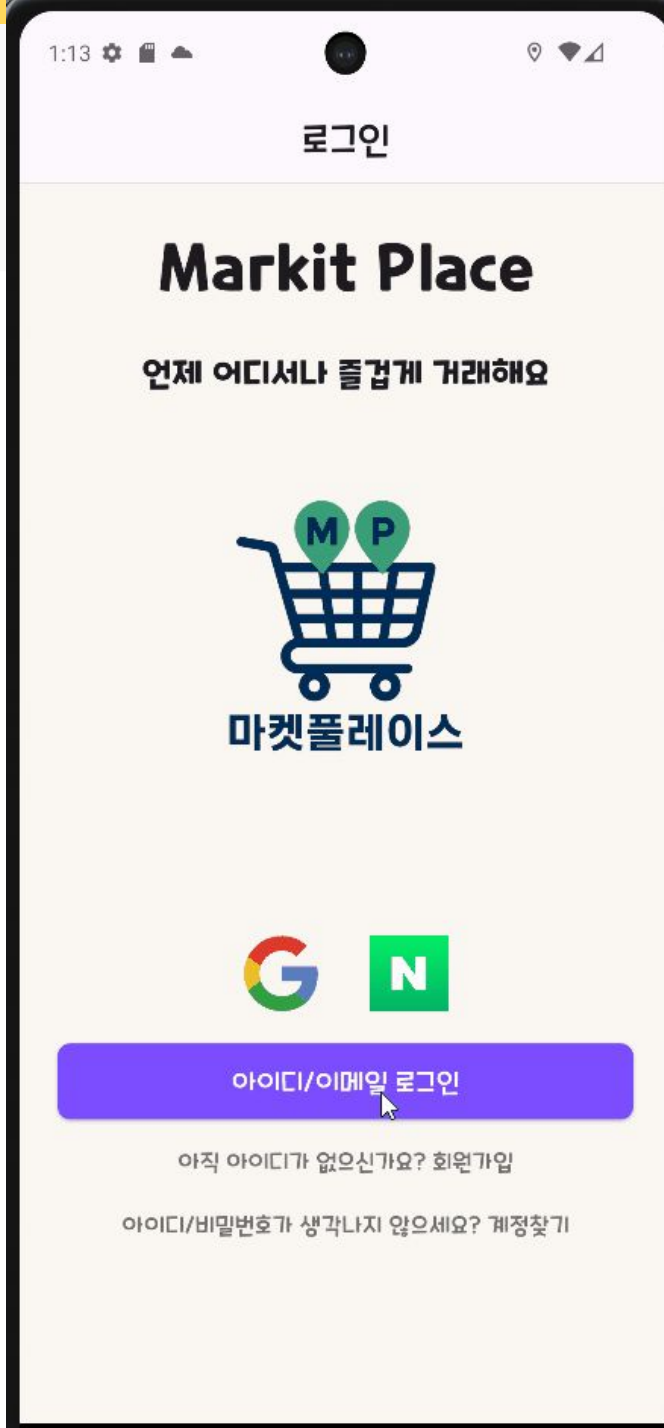
인증번호 입력



04 로그인

계정로그인 / 이메일 로그인 선택

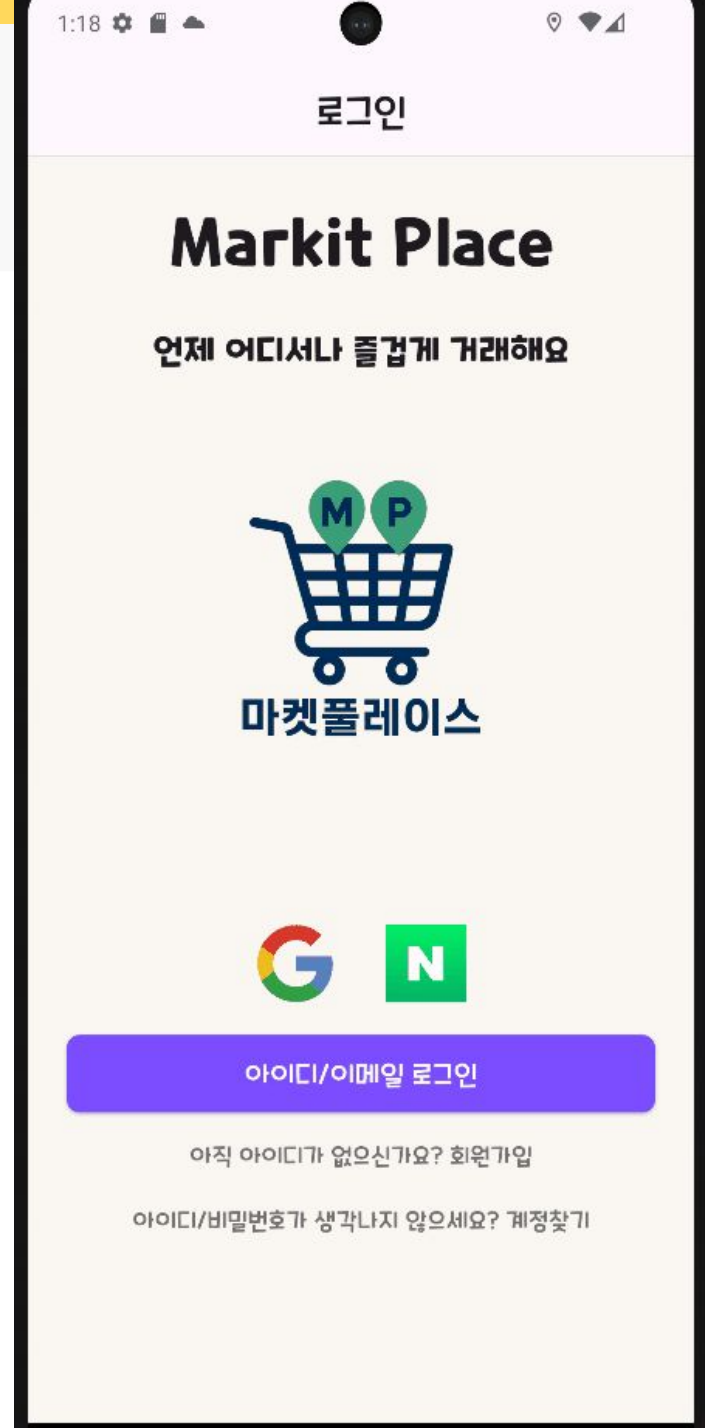
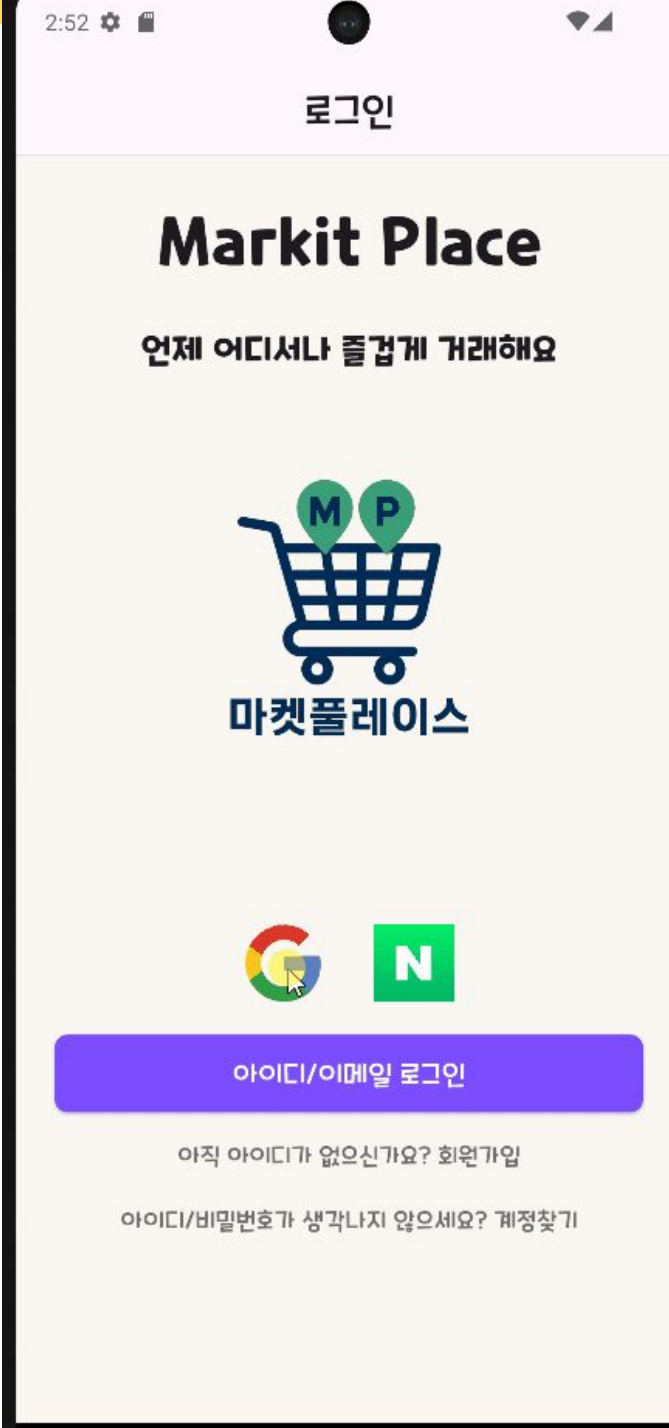
자동 로그인 체크해 자동로그인



04 구글 로그인

구글 계정 선택해 회원가입

구글 계정으로 로그인

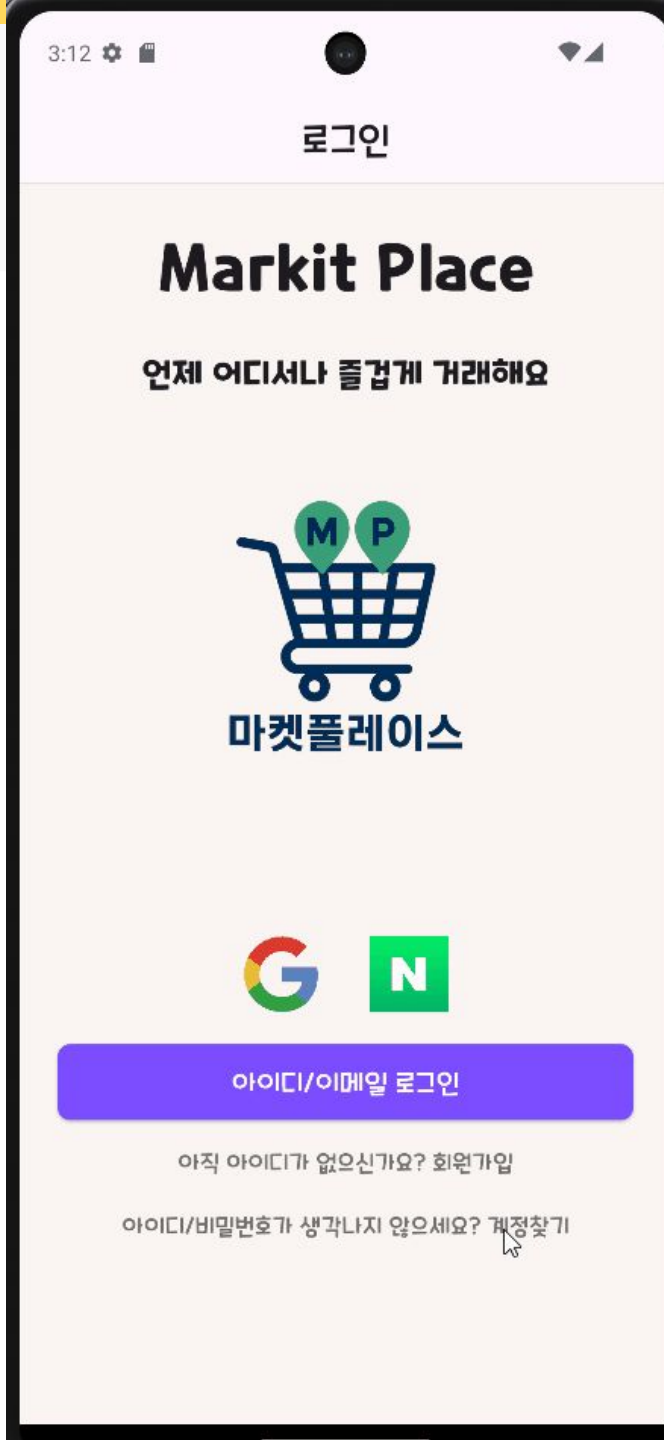


04 계정 찾기

이메일 입력을 통한 아이디찾기

이메일로 완성된 이메일 전송

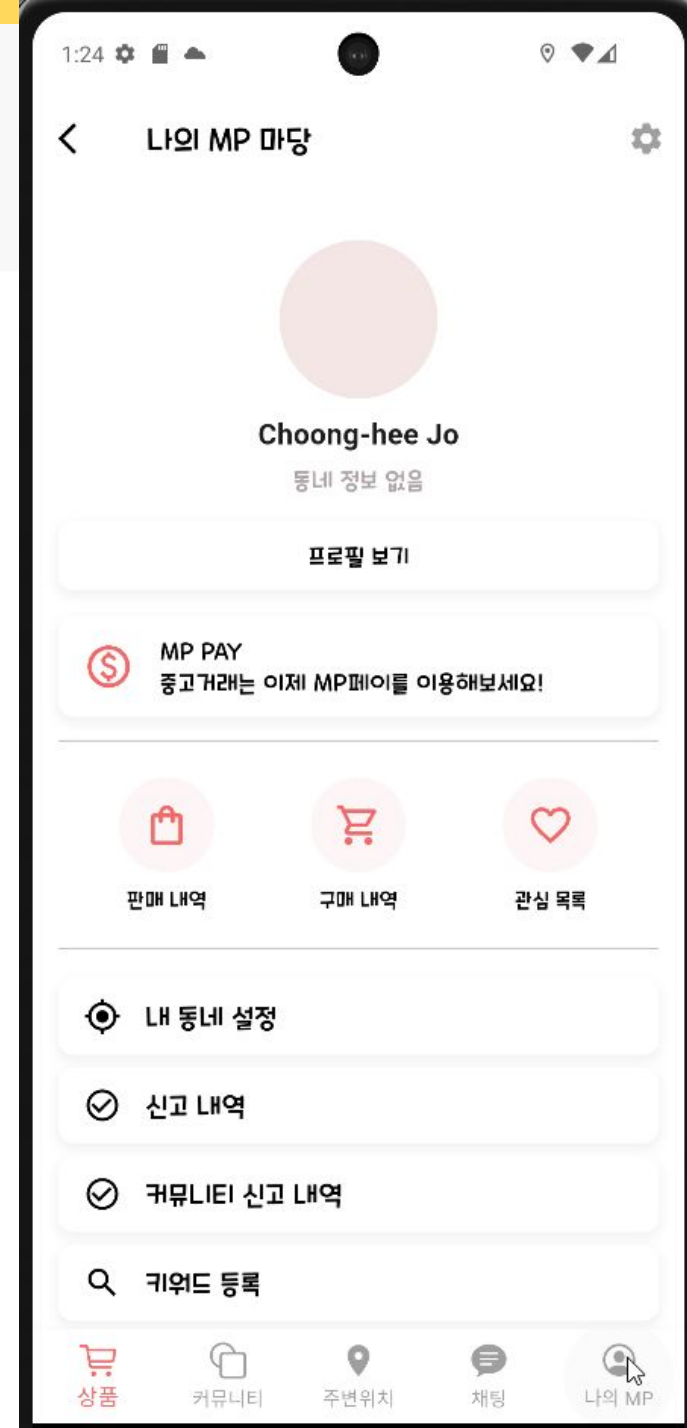
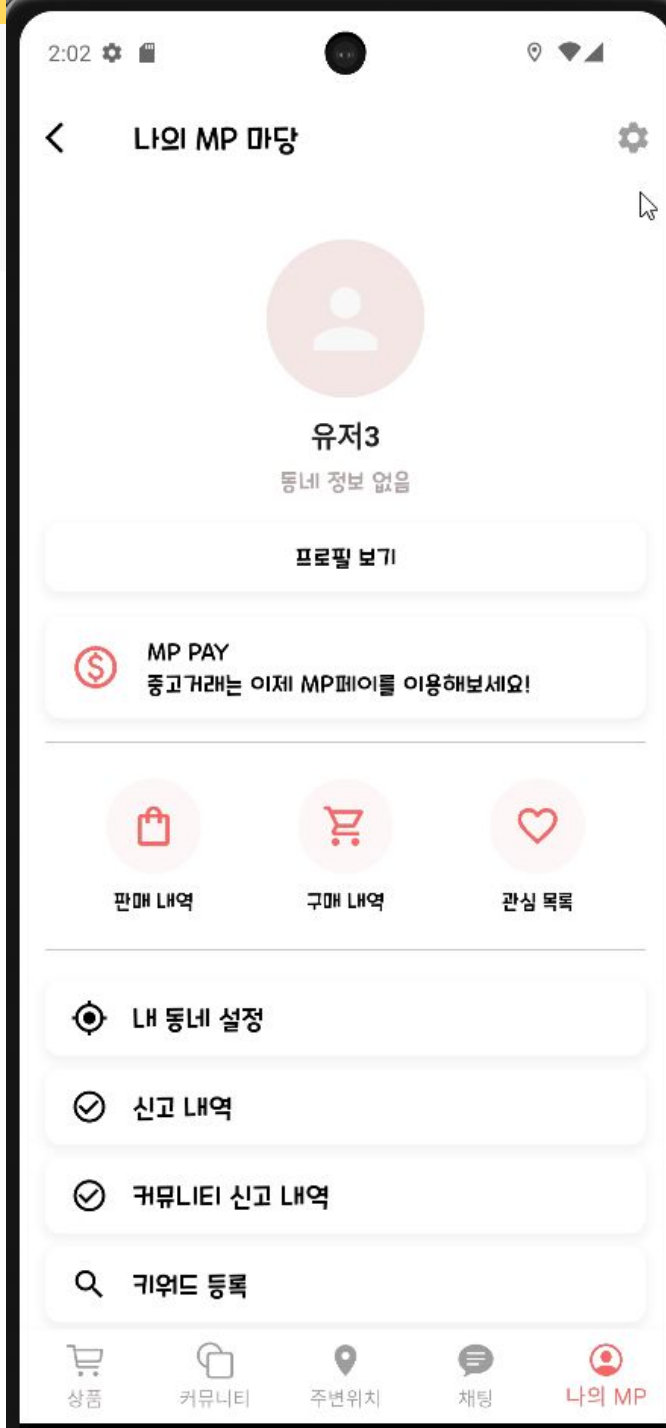
이메일 인증을 통한 비밀번호 리셋



04 프로필 관리

계정 프로필:
닉네임과 사진 변경 가능

소셜 프로필:
소셜계정에 등록된 닉네임과 사진
(수정불가)



04

K-Digital Training

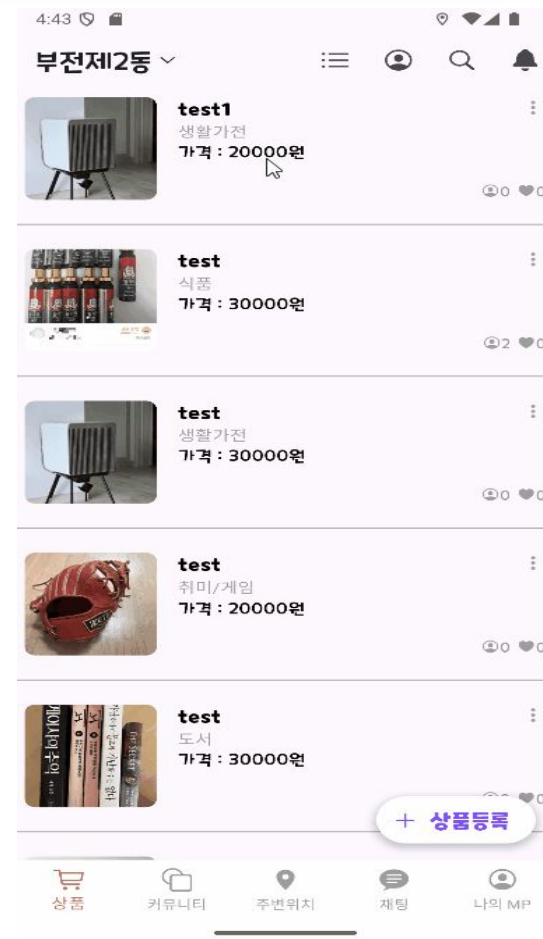
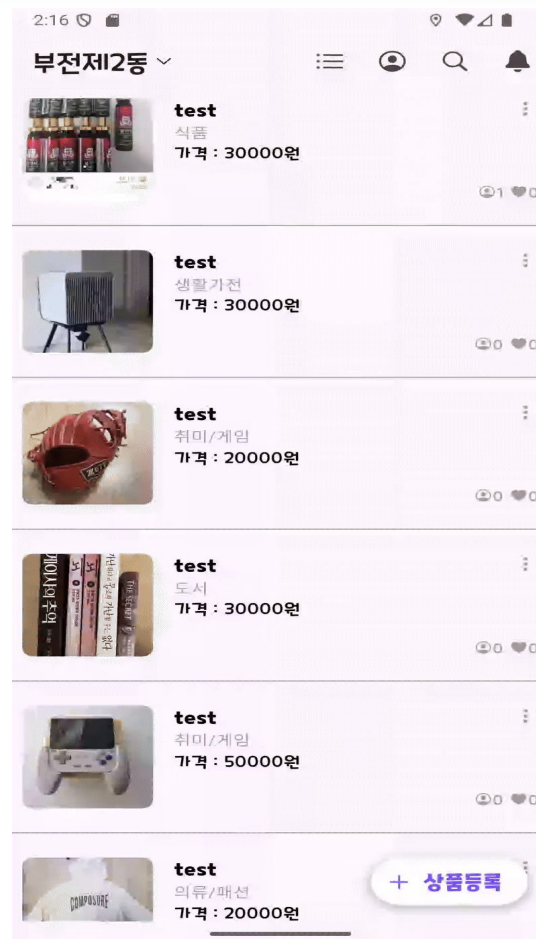
프로젝트 수행 경과

▶ 주요 성과 및 업무



상품 관리 기능

- 상품 등록·수정·삭제 : 안정적 (CRUD) 제공
- 이미지 업로드 : 다중 이미지 지원, 대표 썸네일 자동 지정
- 즐겨찾기 (좋아요) : 실시간 토글 및 카운트 반영
- 표준화된 API : 페이징·정렬·에러 응답 일관 처리



▶ 주요 성과 및 업무

🔍 다중 조건 검색 & 정렬

- 다중 조건 검색(QueryDSL) :
키워드·카테고리·가격·태그·거래 위치
- 정렬 옵션 지원 : 최신순 / 인기순 지원
- 효율적 페이지징 : Pageable + PageImpl
활용
- 성능 최적화 : 필요한 컬럼만 조회, 로딩
전략 개선



04

K-Digital Training

프로젝트 수행 경과

▶ 주요 성과 및 업무

신고 처리 & 제재 자동화

- 신고 승인 → **Event 발행**
(PostRemovalRequestedEvent)
- 리스너 처리 : 글 강제 삭제 & 작성 제한 자동 적용
- 도메인 결함도 최소화 : 정책 변경과 기능 확장에 유연
- 운영 효율화 : 관리자 승인 한 번으로



04

K-Digital Training

프로젝트 수행 경과

▶ 주요 성과 및 업무



제재 정책 & 자동 해제

- **ModerationPolicy** : 신고 횟수별 제재 기간 자동 산출
1회 12시간 → 2회 1일 → 3회 3일 → 4회 7일 → 5+ 영구 정지
- **@Scheduled 스케줄러** : 만료 후 자동 해제
- **운영 효율성** : 수동 해제 불필요, 관리 부담 최소화
- **정책 일관성 & 투명성** : 관리자 화면에서 상태/로그 확인

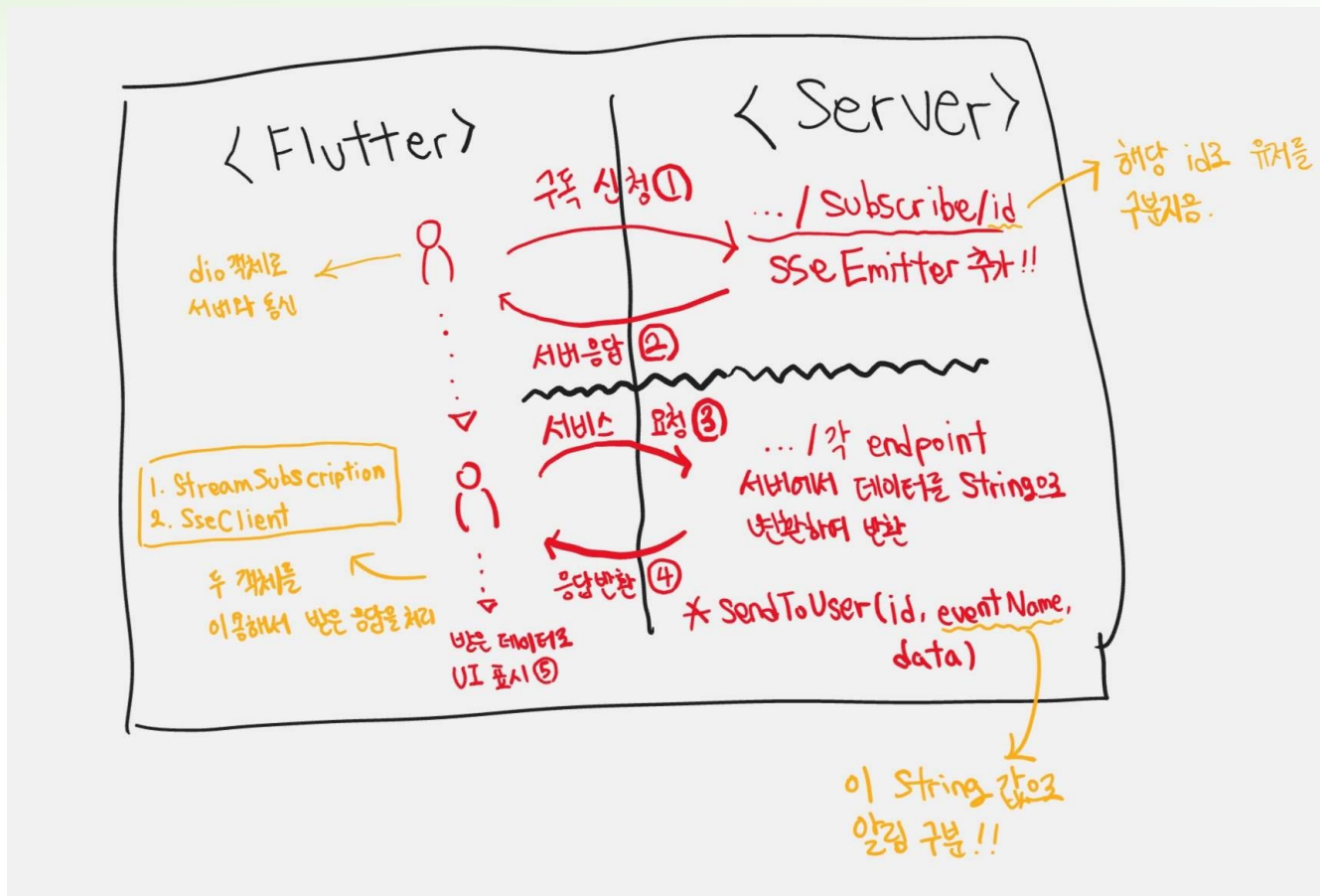
04

SSE(Server-Sent Event)

📢 SSE 기능

통신 흐름 요약

1. Flutter 클라이언트가 **dio**를 사용하여 서버의 **subscribe** 엔드포인트에 특정 **id**로 구독을 요청합니다 (①).
2. 서버는 이 **id**를 기반으로 **sseEmitter** 객체를 추가하고 해당 유저를 식별합니다 (②).
3. 클라이언트는 **StreamSubscription**과 **SseClient**를 사용하여 서버로부터의 응답(이벤트)을 기다립니다 (③).
4. 서버에서 특정 이벤트가 발생하면, 해당 **sseEmitter** 객체를 통해 데이터를 **String** 형태로 변환하여 클라이언트의 **id**로 전송합니다 (④).
5. Flutter 클라이언트는 **StreamSubscription**을 통해 이 데이터를 수신하고 **UI**에 표시합니다 (⑤).



04 AI(Gemini)



Gemini API 사용하기 – Server

```
@Async 1 usage ysb5397 +1
public void askImageForGemini(String userId, GeminiImageRequest geminiRequest) {
    if (apiUrl.trim().isEmpty() || apiKey.trim().isEmpty()) {
        throw new Exception400("API 엔드포인트 또는 API Key가 누락된 잘못된 요청입니다. 설정을 확인해주세요.");
    }
    log.info("서비스 진입");
```

일반 응답 API

```
webClient.post() RequestBodyUriSpec
    .uri( uri: apiUrl + "?key=" + apiKey) RequestBodySpec
    .contentType(MediaType.APPLICATION_JSON)
    .bodyValue(geminiRequest) RequestHeadersSpec<capture of ?>
    .retrieve() ResponseSpec
    .bodyToMono(GeminiImageResponse.class) Mono<GeminiImageResponse>
    .doOnSuccess( GeminiImageResponse response -> {
        log.info("Gemini로부터 성공적인 응답 수신. userId: {}", userId);
        String textResponse = response.extractText();
        if (textResponse != null && !textResponse.isEmpty()) {
            sseUtil.sendToUser(userId, eventName: "AI Response", textResponse);
        } else {
            sseUtil.sendToUser(userId, eventName: "error", data: "AI가 응답을 생성하지 못했습니다.");
        }
    })
// 3. (에러 발생 시) WebClient 통신 중 에러가 나면, 클라이언트에게 에러 메시지를 보낸다.
.doOnError( Throwable error -> {
    log.error("Gemini API 통신 중 에러 발생! userId: {}" userId, error);
```

```
@Async 1 usage ysb5397
public void askImageForGeminiStreaming(String userId, GeminiImageRequest request) {
    if (streamApiUrl.trim().isEmpty() || apiKey.trim().isEmpty()) {
        throw new Exception400("Stream API 엔드포인트 또는 API Key가 누락된 잘못된 요청입니다. 설정을 확인해주세요.");
    }
}
```

스트림 응답 API

```
try {
    webClient.post() RequestBodyUriSpec
        .uri( uri: streamApiUrl + "?key=" + apiKey) RequestBodySpec
        .contentType(MediaType.APPLICATION_JSON)
        .bodyValue(request) RequestHeadersSpec<capture of ?>
        .retrieve() ResponseSpec
        .bodyToFlux(GeminiImageResponse.class) Flux<GeminiImageResponse>
        .subscribe(
            GeminiImageResponse chunk -> {
                String textChunk = chunk.extractText();
                if (textChunk != null && !textChunk.isEmpty()) {
                    sseUtil.sendToUser(userId, eventName: "AI Response", textChunk);
                }
            },
            Throwable error -> {
                sseUtil.sendToUser(userId, eventName: "error", data: "AI 스트리밍 중 오류가 발생했습니다.");
            },
            () -> {
                sseUtil.sendToUser(userId, eventName: "final", data: "stream end"); // -> 스트림이 끝났다는 것
```


04 AI(Gemini)



Gemini API 사용하기 – Flutter

```
Stream<Map<String, dynamic>> subscribe({required int userId}) {  
  final controller = StreamController<Map<String, dynamic>>();  
  
  _streamSubscription?.cancel();  
  
  final url = "$_baseUrl/subscribe/$userId";  
  
  _streamSubscription = SSEClient.subscribeToSSE(  
    method: SSERequestType.GET,  
    url: url,  
    header: {"Accept": "text/event-stream"},  
  ).listen(  
    (event) {  
      final eventName = event.event ?? '';  
  
      if (eventName == 'connect') {  
        controller.add(  
          {"eventName": eventName, "errorMessage": null, "data": null});  
      } else if (eventName == 'AI Response') {  
        if (event.data!.isNotEmpty) {  
          controller.add({  
            "eventName": eventName,  
            "errorMessage": null,  
            "data": event.data  
          });  
          _streamSubscription?.cancel();  
          controller.close();  
        }  
      }  
    },  
    onError: (error) {  
      controller.addError({"errorMessage": "SSE 스트림 오류 발생: $error"});  
      _streamSubscription?.cancel();  
      controller.close();  
    }  
  );  
}
```

응답 처리 로직

```
Future<void> sendImagesForGemini(  
  {required List<XFile> images, required int userId}) async {  
  final List<Map<String, dynamic>> base64Images =  
    await _convertBase64Images(images);  
  
  final List<Map<String, dynamic>> imageParts = base64Images.map((imgMap) {  
    return {  
      "inline_data": {  
        "mime_type": imgMap["mimeType"],  
        "data": imgMap["imageData"]  
      }  
    };  
  }).toList();  
  
  final textPart = {  
    "text":  
      "너는 상품의 장점을 매력적으로 어필하는 전문 마케터야. 제시된 이미지들을 분석해서, 고객이 이 상품을 구매했을 때 얻게 될 경험이나 혜택을 중시"  
  };  
  
  final requestData = {  
    "contents": [  
      {  
        "parts": [  
          ...imageParts,  
          textPart,  
        ]  
      }  
    ]  
  };  
  
  final requestJson = json.encode(requestData);  
  await dio.post("$_baseUrl/image/$userId", data: requestJson);  
}
```

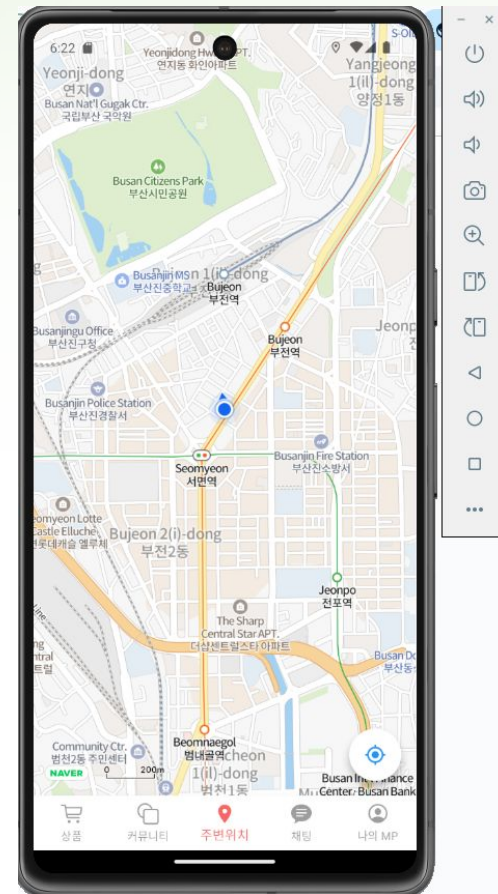
Request 객체 생성

04 Naver Map

Naver Map

1. none, follow, face의 세 가지 모드로 구성
2. none 모드는 오로지 지도를 손가락으로 움직일 때만 되며, 다른 두 모드는 버튼으로 조작 가능
3. 그리고 처음 지도화면 진입시 타 지도처럼 follow 모드로 진입

```
void cycleTrackingMode() {  
    if (_controller == null) return;  
  
    final NLocationTrackingMode nextMode;  
  
    if (state == NLocationTrackingMode.follow) {  
        nextMode = NLocationTrackingMode.face;  
    } else if (state == NLocationTrackingMode.face) {  
        nextMode = NLocationTrackingMode.follow;  
    } else {  
        nextMode = NLocationTrackingMode.follow;  
    }  
  
    _controller!.setLocationTrackingMode(nextMode);  
    state = nextMode;  
}
```



▶ 주요 업무 기능 및 성과

 **Notice**(공지사항)같은 경우 관리자가 공지를 올릴 수 있는 기능으로 공지사항의 관리 **API**를 만들었습니다. 이는 공지사항의 **CRUD**를 제공합니다.

세부적으로는 **RESTful API**의 설계로, 직관적인 **URL** 설계로서 **API** 사용성을 높입니다, **Pagination**의 구현은 공지사항이 쌓일 경우 페이지 단위로 효율적으로 체크할 수 있습니다.

다음으로 권한 제어 기능은 관리자와 유저를 나누고 둘의 권한을 나눕니다.

 **Q&A**기능은 유저가 질문을 올리면 관리자가 응답해 줄 수 있는 기능으로 **Q&A**같은 경우도 관리 **API**와 **CRUD**를 만들었습니다.

세부적으로도 거의 같은 기능으로 **RESTful**설계로 직관적인 **URL** 설계 및 **API**사용성을 높입니다.

Pagination구현 및 권한 제어 기능이 있습니다.

다만 공지사항과 다르게 유저가 글을 쓸 수 있기에 글을 쓴 유저본인 및 관리자가 수정,삭제도 가능합니다.

 **답변(Reply)**은 **Q&A**에 종속된 핵심 하위 기능으로, 질문에 대한 **CRUD**를 가능하게 합니다. 우리는 **Q&A**와 동일한 **RESTful API** 및 **Pagination** 원칙을 적용하여 이 기능을 구축했습니다.

이러한 종속성은 **URL** 구조를 통해 명확히 드러냅니다.

POST : /api/qnas/{qnald}/replies: 답변 생성

GET : /api/qnas/{qnald}/replies/{replyId}: 답변 조회

PUT : /api/qnas/{qnald}/replies/{replyId}: 답변 수정

DELETE : /api/qnas/{qnald}/replies/{replyId}: 답변 삭제

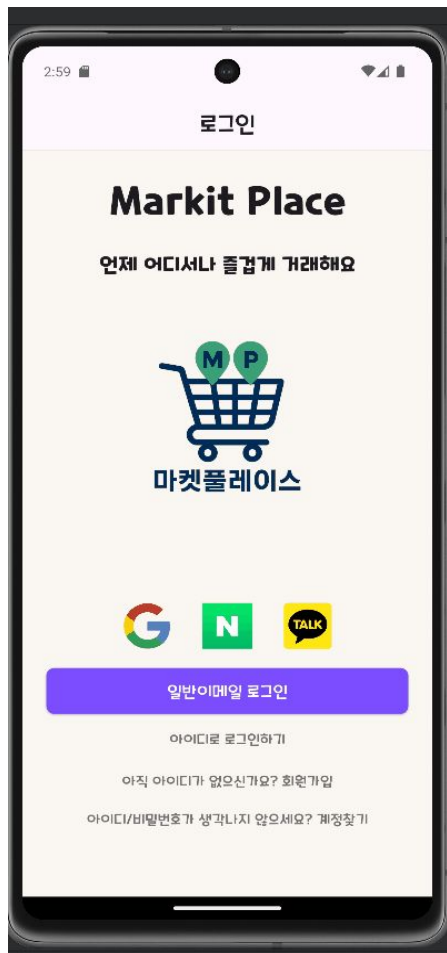
와 같이 **URL**에 부모 자원인 **qnas**의 **ID**를 포함함으로써, 데이터 간의 계층적 관계를 직관적으로 나타내며 **API**의 확장성과 안정성을 높여줍니다

04

K-Digital Training 시연 영상

▶ 주요 업무 및 성과

- Flutter Front 영상



- SpringBoot Server 영상

시나리오 영상

1.회원가입

METHOD: POST SCHEME // HOST [":" PORT] [PATH ["?" QUERY]]

http://localhost:8080/api/members/register

length: 42 byte(s)

QUERY PARAMETERS

HEADERS

Content-Type: application/json

Form

BODY

```
{
  "loginId": "string",
  "password": "Qs$zuUg2Xhb1",
  "email": "string@naver.com",
  "address": "string",
  "isEmailVerified": true,
  "agreedTermIds": [
    1,2,3,4
  ]
}
```

length: 174 bytes

Response

400

Elapsed Time: 8ms

HEADERS

Content-Type: application/json; charset=UTF-8

Transfer-Encoding: chunked

Date: Thu, 18 Sep 2025 03:27:58 GMT ~27s

Connection: close

BODY

```
{
  "success": false,
  "response": null,
  "error": ...
}
```

Top Bottom Collapse Open 2Request Copy Download

▶ 프로젝트 결과물에 대한 완성도 평가

전체 기능

- 회원가입 및 로그인 구현
- 중고 거래 게시판 (수정, 삭제, 댓글, 좋아요)
- 채팅 기능 (1:1 실시간 채팅)
- 커뮤니티 (동네생활, 카테고리, 댓글, 좋아요)
- 거래관리 (판매 상태 표시)
- 거래 후기 작성
- 공지사항 (어드민 만 작성가능)
- Q&A 작성 가능
- 검색 및 필터 고도화 (커뮤니티, 중고거래) 공용
- 사용자 평판 시스템
- 신고 기능 (거래 게시판, 커뮤니티)
- 동네인증 GPS (현재 위치로 부터 동네 설정 가능, 임의로 설정 가능)
- 결제 기능 (물건에 관해서 구매시 결제가능하게)
- 알림 기능 (자신의 게시글 좋아요 및 관련 상품에 관해서 알림)

추후 개선점이나 보완할 점

UI/UX 완성도 향상

앱 느낌을 더 살리기 위해 일관된 색상, 폰트, 아이콘 사용

버튼, 카드, 리스트 등 컴포넌트 간 여백과 비율 조정

반응형 레이아웃 고려 (다른 기기 해상도에서도 자연스럽게 보이도록)

기능 개선

출석 체크 기능 구현 → 회원별 포인트 적립

알람/푸시 기능 추가 → 출석 체크 알림, 포인트 지급 안내

테스트 및 안정성

단위 테스트 (JUnit, Flutter 테스트) 작성

에러 로그 모니터링 및 개선 (Sentry 등)

자체 평가 의견

▶ 개인 또는 우리 팀이 **잘한 부분과 아쉬운 점** 및 깨달은 점 성과

손지윤

잘한 점 : 초기에 계획했던 각자 맡은 기능을 빠짐없이 구현하였고, 더 필요한 추가 적인 기능도 만들어 낼 수 있었습니다.

아쉬운 점 : 테스트를 하는 과정 중 오류 발생으로 코드를 수정하는 일이 잦았습니다. 프로젝트 진행 중 테스트 작업을 소홀히 하지 않아야겠습니다.

양성빈

아쉬운 점은 딱히 없었던 것 같고, 좋았던 점은 여러 API를 계속 이용해서 써볼 수 있었던 것 같아서 상당히 괜찮았던 것 같습니다

황지백

잘한 점 : 솔직히 제 기준에서 어렵다고 느껴졌던 부분들이 많았는데 그래도 열심히해서 해낸부분이 있던것이 잘한것 같습니다.

아쉬운 점 : 해결능력이 좀 투박하고 기능을 하나 만드는데 이해하는 시간이 오래걸려 작업 속도가 느렸습니다.다음에는 좀 더 생산력을 늘리고 유연한 해결능력을 가지려고 노력해 보고싶습니다.

자체 평가 의견

▶ 개인 또는 우리 팀이 **잘한 부분과 아쉬운 점** 및 깨달은 점 성과

조정우

잘한 점 : 각자 맡은 일에 최선을 다하시고 부족한 부분없게 100% 일을 해주셔서 많은 도움이 되었습니다

아쉬운 점 : 팀원은 아니라 제가 개발하는 과정에서 너무 한가지의 기능에 집중을 하다보니 다른 기능을 만들지 못하고 작업에 진전이 없어서 아쉬웠습니다

유류진

잘한 점 : 최대한 할 수 있을 만큼 욕심 내지 않고 각자 스스로 맡은 바 잘해주었습니다

아쉬운 점 : 개인적으로 UI 디자인을 조금 더 신경 써 보고 싶었는데 시간이 부족해서 하지 못했습니다

조충희

잘한 점 : 팀원들이 밤낮을 가리지 않고 노력해서 높은 품질의 제품을 만들 수 있었다고 생각한다.

아쉬운 점 : 초기 테이블과 기능 설계에 좀더 많은 논의를 하지 못한 점이 아쉽다.

조현진

잘한 점 : 파이널 프로젝트 답게 성장한 모습을 보여주며 서로가 할 수 있는 일을 각자 잘 보여 주었다

아쉬운 점 : 각자 맡은 일들을 하다 보니 소통 부족이 되어 생긴 문제가 있었다. 그러다 보니 좀 더 많은 기술들을 만들 수 있을 것 같았는데 못 만들었던 것이 아쉬웠다